

Research Notes on Adagrad optimizer

Adagrad optimizer

>> Section 1

Implicit updates (ISGD) As mentioned earlier, classical stochastic gradient descent is generally sensitive to learning rate η . Fast convergence requires large learning rates but this may induce numerical instability. The problem can be largely solved by considering implicit updates whereby the stochastic gradient is evaluated at the next iterate rather than the current one:

>> Section 2

Further reading Bottou, Léon (2004), "Stochastic Learning", Advanced Lectures on Machine Learning, LNAI, vol. 3176, Springer, pp. 146–168, ISBN 978-3-540-23122-6 Buduma, Nikhil; Locascio, Nicholas (2017), "Beyond Gradient Descent", Fundamentals of Deep Learning : Designing Next-Generation Machine Intelligence Algorithms, O'Reilly, ISBN 9781491925584 LeCun, Yann A.; Bottou, Léon; Orr, Genevieve B.; Müller, Klaus-Robert (2012), "Efficient BackProp", Neural Networks: Tricks of the Trade, Springer, pp. 9–48, ISBN 978-3-642-35288-1 Spall, James C. (2003), Introduction to Stochastic Search and Optimization, Wiley, ISBN 978-0-471-33052-3

>> Section 3

The step size is denoted by η (sometimes called the learning rate in machine learning) and here " $:=$ " denotes the update of a variable in the algorithm. In many cases, the summand functions have a simple form that enables inexpensive evaluations of the sum-function and the sum gradient. For example, in statistics, one-parameter exponential families allow economical function-evaluations and gradient-evaluations. However, in other cases, evaluating the sum-gradient may require expensive evaluations of the gradients from all summand functions. When the training set is enormous and no simple formulas exist, evaluating the sums of gradients becomes very expensive, because evaluating the gradient requires evaluating all the summand functions' gradients. To economize on the computational cost at every iteration, stochastic gradient descent samples a subset of summand functions at every step. This is very effective in the case of large-scale machine learning problems.

>> Section 4

$w := w - \eta \nabla Q(w) = w - \eta \sum_{i=1}^n \nabla Q_i(w)$.
$$w := w - \eta \sum_{i=1}^n \nabla Q_i(w)$$

>> Section 5

$w_{\text{new}} = w_{\text{old}} + \frac{\eta}{\sqrt{\sum_{i=1}^t \|g_{\tau,j}\|^2}} (y_i - x_i' w_{\text{old}}) x_i$.
$$w^{\text{new}} = w^{\text{old}} + \frac{\eta}{\sqrt{\sum_{i=1}^t \|g_{\tau,j}\|^2}} (y_i - x_i' w^{\text{old}}) x_i$$

>> Section 6

This equation is implicit since w_{new} appears on both sides of the equation. It is a stochastic form of the proximal gradient method since the update can also be written as:

>> Section 7

Second-order methods A stochastic analogue of the standard (deterministic) Newton–Raphson algorithm (a "second-order" method) provides an asymptotically optimal or near-optimal form of iterative optimization in the setting of stochastic approximation. A method that uses direct measurements of the Hessian matrices of the summands in the empirical risk function was developed by Byrd, Hansen, Nocedal, and Singer. However, directly determining the required Hessian matrices for optimization may not be possible in practice. Practical and theoretically sound methods for second-order versions of SGD that do not require direct Hessian information are given by Spall and others. (A less efficient method based on finite differences, instead of simultaneous perturbations, is given by Ruppert.) Another approach to the approximation Hessian matrix is replacing it with the Fisher information matrix, which transforms usual gradient to natural. These methods not requiring direct Hessian information are based on either values of the summands in the above empirical risk function or values of the gradients of the summands (i.e., the SGD inputs). In particular, second-order optimality is asymptotically achievable without direct calculation of the Hessian matrices of the summands in the empirical risk function. When the objective is a nonlinear least-squares loss

>> Section 8

$G_{j,j} = \sum_{\tau=1}^t g_{\tau,j}^2$.
$$G_{j,j} = \sum_{\tau=1}^t g_{\tau,j}^2$$
 This vector essentially stores a historical sum of gradient squares by dimension and is updated after every iteration. The formula for an update is now

>> Section 9

$\bar{w} = \frac{1}{t} \sum_{i=0}^{t-1} w_i$.
$$\bar{w} = \frac{1}{t} \sum_{i=0}^{t-1} w_i$$
 When optimization is done, this averaged parameter vector takes the place of w .

>> Section 10

History In 1951, Herbert Robbins and Sutton Monro introduced the earliest stochastic approximation methods,

Research Notes on Adagrad optimizer

Adagrad optimizer

preceding stochastic gradient descent. Building on this work one year later, Jack Kiefer and Jacob Wolfowitz published an optimization algorithm very close to stochastic gradient descent, using central differences as an approximation of the gradient. Later in the 1950s, Frank Rosenblatt used SGD to optimize his perceptron model, demonstrating the first applicability of stochastic gradient descent to neural networks. Backpropagation was first described in 1986, with stochastic gradient descent being used to efficiently optimize parameters across neural networks with multiple hidden layers. Soon after, another improvement was developed: mini-batch gradient descent, where small batches of data are substituted for single samples. In 1997, the practical performance benefits from vectorization achievable with such small batches were first explored, paving the way for efficient optimization in machine learning. As of 2023, this mini-batch approach remains the norm for training neural networks, balancing the benefits of stochastic gradient descent with gradient descent. By the 1980s, momentum had already been introduced, and was added to SGD optimization techniques in 1986. However, these optimization techniques assumed constant hyperparameters, i.e. a fixed learning rate and momentum parameter. In the 2010s, adaptive approaches to applying SGD with a per-parameter learning rate were introduced with AdaGrad (for "Adaptive Gradient") in 2011 and RMSprop (for "Root Mean Square Propagation") in 2012. In 2014, Adam (for "Adaptive Moment Estimation") was published, applying the adaptive approaches of RMSprop to momentum; many improvements and branches of Adam were then developed such as Adadelta, Adagrad, AdamW, and Adamax. Within machine learning, approaches to optimization in 2023 are dominated by Adam-derived optimizers, TensorFlow and PyTorch, by far the most popular machine learning libraries, as of 2023 largely only include Adam-derived optimizers, as well as predecessors to Adam such as RMSprop and classic SGD. PyTorch also partially supports limited-memory BFGS, a line-search method, but only for single-device setups without parameter groups.

>> Section 11

RMSProp has shown good adaptation of learning rate in different applications. RMSProp can be seen as a generalization of Rprop and is capable to work with mini-batches as well opposed to only full-batches.